ASDXDDL8M3 Data Modelling and Management Day 1 Summary

Monday, October 26th, 2026

Overview

Key participants: QA Trainer leads the session and provides the teaching and guidance; learners contributing include Iain Sim, Nicola Donnelly, Nobin Perumpally John and others grouped into four teams for the movie modeling task.

Main topics and outcomes: The session reinforces why single-sheet Excel designs fail and teaches practical remedies through relational modeling and normalization, then moves into concrete data-type guidance required to implement schemas. The trainer covers keys (primary and surrogate), table separation, foreign keys, ER diagrams, anomaly types, and the three normal forms (1NF–3NF), and assigns a movie-database modeling exercise. In the transcript segment, the trainer gives an extensive walkthrough of SQL Server data types—integers (tinyint to bigint), varchar/nvarchar, decimal/numeric with precision and scale, float/real approximations, money/smallmoney, bit for booleans, and date/time variants—and ties each type to dataset attributes like names, budgets, vote averages, popularity, runtimes, and release dates.

Decisions and commitments: QA Trainer assigns the movie modeling deliverables: identify entities (movie, actor, director), propose at least two bridge tables (actor-movie-character and director-movie), produce an ERD, and prepare a data dictionary that lists attributes, SQL data types, and constraints. The trainer instructs teams to create surrogate primary keys where none exist, to mark required attributes as NOT NULL, and to retain attribute names from the collected dataset. QA Trainer also commits to further, focused data-type instruction first thing in the next session and to continue the ERD and modeling work then.

Logistics and follow-ups: QA Trainer provides lab access details, commits to arranging additional cloud training keys for learners who cannot connect, and will capture evidence and group work on the collab board tomorrow so absent students can catch up. Teams are expected to inspect the Excel data, propose data types and constraints in their data dictionaries, and finalize ERDs in the next session.

Detailed Summary

Limitations of Excel

Trainer walks through why Excel alone becomes problematic as data grows: it exposes sensitive information, creates redundant entries, and struggles with ambiguity when similar names appear. The scenario of a small language school

illustrates how storing employee, car, and salary data in one spreadsheet leads to privacy and update anomalies. The current state: Excel can work for initial small datasets but introduces security, redundancy, and maintainability risks as the business scales.

- QA Trainer explains that storing employee details, salary, and car data in one Excel file exposes personal information to anyone with access.
- QA Trainer notes that Walter (a warden) could see salary and contact details in a shared Excel file, which raises confidentiality concerns.
- QA Trainer describes the update anomaly where James appears twice for two cars and updating one row but not the duplicate causes inconsistent data.
- QA Trainer points out that similar names like two Simon Jenkins create ambiguity without unique identifiers in Excel.

Unique Identifiers and Keys

Participants discuss introducing primary keys to uniquely identify records and avoid ambiguity. The trainer endorses using an employee ID as a primary key and explains how it distinguishes identical names and prevents duplicate-person records. The current state: using unique IDs is recommended as a first mitigation step but does not fully solve redundancy across related data.

- Iain Sim suggests using an employee number as a primary key to uniquely identify each person.
- QA Trainer confirms that a primary key like employee ID prevents confusion between identical names and helps find the correct record.
- QA Trainer clarifies that primary keys must be unique and cannot repeat across rows in a table.
- QA Trainer explains surrogate keys are common and may include meaningful prefixes (e.g., location codes) to convey context.

Separating Data into Tables

The group proposes splitting related data into separate tables (entities) to reduce redundancy and control access. The trainer recommends separate employee and car tables and potentially a separate salary table to control HR access. The current state: moving from a single Excel sheet to multiple structured tables reduces exposure and repetition and enables controlled sharing via foreign keys.

- Nicola Donnelly proposes a car details table to ensure Walter can see car data without personal or salary details.
- QA Trainer explains that separating employee and car information into distinct tables reduces redundant personal data entries and supports restricted access.

- QA Trainer suggests placing salary data in a separate table if access must be limited to HR.

Foreign Keys and Relationships

Trainer explains foreign keys as the mechanism linking tables: the employee ID in the car table becomes a foreign key referencing the employee table. This approach allows repeating the foreign key (e.g., multiple cars for one employee) without repeating full personal details. The current state: using primary and foreign key relationships addresses redundancy and supports controlled access, integrity, and availability.

- QA Trainer defines the employee ID in the car table as a foreign key that links back to the employee table.
- QA Trainer states that foreign keys can repeat in a table to represent one-to-many relationships, such as multiple cars for one employee.
- QA Trainer frames this approach as improving confidentiality, integrity, and availability (CIA) of data by limiting what each user can access.

Basic SQL Querying

Trainer demonstrates a simple SQL SELECT with JOIN and WHERE clauses to retrieve car registration plus employee info by joining car and employee tables on employee ID and filtering for a specific employee. This shows how relational databases let you select only required columns and apply filters, unlike a flat Excel file. The current state: participants are introduced to the idea of querying joined tables to get targeted results.

- QA Trainer outlines a SELECT statement that picks reg number, employee ID, and first name from joined car and employee tables.
- QA Trainer explains using an ON clause to match employee IDs between tables and a WHERE clause to filter for a specific employee.
- QA Trainer emphasizes that choosing specific columns in SQL avoids exposing unnecessary personal information.

Introduction to Data Modeling

Trainer shifts focus to data modeling: identifying what data exists, grouping related attributes into entities (tables), and mapping relationships between entities. Examples include electricity bills, movies, and supermarket receipts to illustrate how different pieces of information come from distinct tables (customer, meter, tariff, product). The current state: participants gain a practical sense of identifying entities and where attributes belong before building a database.

- QA Trainer asks participants to identify entities and attributes from real-world examples like electricity bills, demonstrating customer, meter, tariff, usage, and invoice as separate entities.

- QA Trainer uses movie and supermarket examples to show director, actor, genre, product, price, quantity, and category as attributes sourced from different tables.
- QA Trainer stresses that understanding what data you have is a prerequisite to deciding entities and attributes for modeling.

Purpose of Analysis in Modeling

Trainer explains that analysis examines existing data to design structures that support business functions, reduce redundancy, and enable planning (e.g., seasonal product placement). Analysis identifies which attributes are required, how to collect them once, and how to order and store data for future queries. The current state: analysis is positioned as the step before modeling and normalization to ensure data quality and usefulness.

- QA Trainer says analysis helps determine which data is required and how it impacts outcomes like inventory or marketing decisions.
- QA Trainer highlights that correct data collection prevents redundancy and anomalies and enables high-level planning.
- QA Trainer emphasizes that analysis informs entity identification and attribute selection before building a model.

Entity-Relationship Concepts

Trainer covers ERD basics: entities (people, products, events, places), attributes that describe entities, and relationships (one-to-many, many-to-many). Airline, banking, and retail examples illustrate different relationships and the need for bridge tables where appropriate. The current state: participants receive guidance on mapping entities and relationships and using ER diagrams to represent data structure.

- QA Trainer defines entities as nouns such as customer, account, product, flight, and aircraft and explains that attributes describe those entities.
- QA Trainer explains relationships using airline and banking examples to illustrate one-to-many and many-to-many patterns.
- QA Trainer recommends drawing ER diagrams to visualize how entities link and where bridge tables may be needed.

Modeling Strategy and Stages

Trainer outlines the modeling process: collect data, analyze it, identify entities, list attributes, normalize/clean data, and produce logical/physical models. The trainer emphasizes mapping invoice fields to entities (customer, order, product) and designing mandatory versus optional attributes. The current state: participants receive a practical, staged workflow to prepare raw data for a relational database.

- QA Trainer lists stages: gather data, analyze it, identify entities, normalize the data, and design the logical model.

- QA Trainer uses an invoice to show how order, customer, product, salesperson, and quantity attributes come from different tables.
- QA Trainer stresses separating mandatory (NOT NULL) attributes from optional ones to ensure data completeness where required.

Attributes and Data Quality

Trainer clarifies the difference between entities and attributes, discusses attribute rules (atomicity, single value per attribute), composite attributes like addresses, and enforcing data types/constraints. Ensuring attributes depend on the key and collecting attributes in atomic columns facilitates filtering and analysis. The current state: participants understand attribute design principles to avoid multi-valued or improperly formatted fields.

- QA Trainer states that entities are objects and attributes are properties that describe those objects, and attributes should use nouns.
- QA Trainer explains that each attribute should hold one atomic value and that composite attributes (e.g., address) should be broken into separate columns.
- QA Trainer advises enforcing data types and NOT NULL constraints for mandatory attributes and allowing NULL for optional ones.

Entities and Examples Exercise

Trainer runs a practical exercise asking learners to identify entities and attributes for supermarket products, employees, patients, and students to reinforce modeling decisions. The exercise shows how some attributes belong locally while others should be foreign keys linking to separate tables (e.g., category, department, GP). The current state: participants practice mapping keys and attributes and recognizing when to extract attributes to separate entities.

- QA Trainer asks learners to define product attributes (e.g., barcode as key, description, brand, category, unit price) and identifies category as a candidate foreign key.
- QA Trainer guides learners to model employee attributes (employee ID key, name, department, manager, contract type) and identifies department and manager as foreign-key relationships.
- QA Trainer shows patient modeling using a national ID (CHI/NHS) as primary key and identifies doctor, surgery branch, and pharmacy as related foreign-key entities.
- QA Trainer models students with SID as key and identifies course, tutor, campus, funding status, and level as attributes or foreign-key links.

Entity-Relationship Diagrams

Trainer explains ER diagrams as the visual step that follows entity identification: relationships are drawn first (mandatory vs optional), then attributes are added. ERDs help surface required foreign keys, mandatory relationships (NOT NULL), optional relationships, and conditional relationships that depend on business processes. The current state: participants are advised to iterate on ERDs and refine them as new data arrives.

- QA Trainer explains that ERDs let you join entities and reveal which attributes belong locally versus coming from related entities.
- QA Trainer distinguishes mandatory relationships (e.g., order must have a product) from optional or conditional ones (e.g., employee project assignment).
- QA Trainer recommends iterating and refining ER diagrams as requirements and data change rather than seeing them as fixed designs.

One-to-One and One-to-Many

Trainer outlines relationship cardinality: one-to-one examples (order-invoice, payment-invoice) and one-to-many examples (department to employees, surgery to patients). Understanding cardinality determines where foreign keys are required and whether they must be NOT NULL. The current state: participants can classify relationships and enforce constraints accordingly.

- QA Trainer uses order/invoice and payment/invoice to illustrate one-to-one relationships where each order has a single invoice.
- QA Trainer uses department-to-employee and surgery-to-patient as examples of common one-to-many relationships.
- QA Trainer notes that mandatory one-to-many relationships should enforce NOT NULL foreign keys in the child table.

Many-to-Many and Bridge Tables

Trainer covers the many-to-many problem: when two entities relate to each other in both directions (e.g., employees-projects, students-classes), storing relations directly in either table creates redundancy and breaks normalization. The trainer prescribes bridge (junction) tables that contain the primary keys of both entities as foreign keys to resolve many-to-many and enable efficient querying. The current state: participants understand that bridging tables are required to model many-to-many correctly.

- QA Trainer explains that many-to-many relationships (e.g., employee-project, student-class, parts-warehouse) cause redundancy if recorded directly in entity tables and must be broken with a bridge table.
- QA Trainer prescribes creating a bridge table with foreign keys to each primary table (e.g., student_class_bridge) to allow correct querying and maintain normalization.

- QA Trainer highlights that bridge tables enable queries like 'which employees work on project A' or 'which warehouses stock part X' without duplicating entity records.
- QA Trainer shows that Excel's composite or comma-separated fields cannot be reliably filtered and are a clear sign normalization is needed.

Modeling Relationship Types

Trainer explains relationship types in context (mandatory, optional, conditional) and how that affects schema constraints and data collection policies. Examples include desk bookings tied to department and desk IDs, hybrid work scenarios, and conditional entities like operations for some patients. The current state: participants can map business rules to NOT NULL constraints, optional foreign keys, and conditional logic in the schema.

- QA Trainer uses desk booking to show conditional relationships where desk booking requires both employee ID and department ID as NOT NULL foreign keys when booking occurs.
- QA Trainer shows that some relationships are conditional (e.g., not all patients have operations; projects may be optional for employees) and should be modeled accordingly.
- QA Trainer reiterates that mandatory relationships should be enforced at database level to prevent incomplete records.

Normalization Introduction

Trainer introduces normalization as the standard process to structure data, remove redundancy, and enforce consistent collection formats across sources. Normalization is framed as both policy (standards for data capture) and technical transformation to split complex attributes (like addresses) into atomic columns and to separate entities appropriately. The current state: normalization will be covered in depth after the break and is presented as essential for integration and reliable querying.

- QA Trainer defines normalization as the process and policy to structure data and standardize collection to enable integration from multiple sources.
- QA Trainer emphasizes splitting complex attributes (e.g., address) into atomic fields and applying consistent formats for phone numbers and dates.
- QA Trainer uses an invoice example to show how customer, supplier, order, and product entities map into normalized tables for queryable analysis.

Normalization Anomalies

Trainer explains the three key anomalies—update, insertion, and deletion—that normalization addresses to preserve data integrity. Using real examples (duplicated employee rows, ordering before a customer exists, and deleting a person causing loss of related records), the trainer shows why structured tables and keys prevent

these failures. The current state: participants understand anomalies and how normalization reduces inconsistency, missing data problems, and unintended deletions.

- QA Trainer defines update anomaly and demonstrates it with the James example where only one of two duplicate rows is updated, creating inconsistency.
- QA Trainer explains insertion anomaly as a 'catch-22' where an order cannot be recorded until a customer account exists, so related data must be collected first.
- QA Trainer describes deletion anomaly where removing a person (e.g., a doctor) can inadvertently remove related records like appointments, and attributes that deletion risk must be modeled to avoid data loss.

Normal Forms Explained

Trainer covers first, second, and third normal forms with practical rules: 1NF requires atomic values and unique column names; 2NF requires a primary key and that non-key attributes fully depend on the whole key; 3NF removes transitive dependencies so non-key attributes depend only on the key. The current state: learners are asked to bring raw Excel data to 1NF then progress to 2NF and 3NF for relational databases.

- QA Trainer states that first normal form requires atomic column values, consistent data types, and no repeating columns.
- QA Trainer explains second normal form requires a primary key and that non-key attributes must be functionally dependent on the whole key.
- QA Trainer clarifies third normal form requires removal of transitive dependencies and that calculated fields (e.g., order total) should not be stored as base data.

Applying Normalization to Example

Trainer walks through normalizing a messy Excel sales dataset: identifying inconsistent date formats, composite addresses, duplicate product prices, and missing fields; converting it to 1NF, then splitting into customer, order, product, publisher, and an order_item bridge to reach 3NF. The current state: the class works through the provided spreadsheet to produce a 3NF model and a corresponding ER diagram.

- QA Trainer identifies inconsistent date formats and composite address fields in the raw Excel data and directs learners to standardize them for 1NF.
- QA Trainer removes calculated fields like order total and moves quantity into an order-item bridge to satisfy 2NF and 3NF.

- QA Trainer creates distinct tables for customer, order, product, and publisher, and prescribes an order_item bridge table to resolve the many-to-many between orders and products.

Practical Many-to-Many Activity

Learners perform a hands-on activity converting a first-normal-form orders-products table into a normalized design with order, product, and a bridge (order_item) to carry quantity. The trainer coaches groups through choosing keys, assigning attributes to the correct entity, and deciding whether a fourth table (customer) is appropriate. The current state: groups produce normalized table sets that include foreign keys for bridging and confirm how the design supports order fulfillment.

- QA Trainer assigns a many-to-many activity where participants split a flat orders-products table into order, product, and a bridge table to include quantity.
- Learner Nobin proposes the bridge table containing foreign keys for order number and product ID plus quantity, and QA Trainer advises marking those as foreign keys rather than primary keys.
- QA Trainer confirms that either a three-table or four-table design (including a separate customer table) correctly resolves the many-to-many problem and supports order processing.

Group Modeling Exercise Setup

Trainer organizes learners into small groups and assigns a larger movie-database modeling task: convert a complex movie/actor Excel sheet (1NF) to a 3NF relational model, identify entities and bridges, and create a data dictionary. The current state: groups are formed, files are shared, and expectations for deliverables (entities, bridges, relationships, data types) are set.

- QA Trainer divides participants into four groups and assigns the movie scenario modeling exercise using an Excel tab 'movies and actor' and a Word brief.
- QA Trainer instructs learners to identify entities (including at least two bridge tables), derive attributes, and prepare a data dictionary specifying data types.
- QA Trainer states the task focuses on modeling (entities, keys, relationships) rather than re-normalizing the dataset, and expects work to continue into the next session if needed.

Data Types and Data Dictionary

Trainer introduces the need to specify data types during modeling as part of the data dictionary deliverable. Learners must identify appropriate types for attributes (dates, integers, varchars) because those choices inform schema design and data validation.

The current state: the class will capture data types as part of the movie-database modeling activity.

- QA Trainer teaches that identifying data types is a required component of the modeling task and will feed into the data dictionary.
- QA Trainer advises mapping attributes to appropriate types (e.g., date, integer, varchar) to ensure consistency and enable database validation.
- QA Trainer defines the data dictionary as documentation that lists entities, attributes, data types, and constraints so future maintainers can understand the database.

Lab Access and Logistics

Trainer checks learner access to remote labs and provides step-by-step guidance for connecting to allocated desktops, sharing usernames, passwords, and access codes via direct messages and chat. The trainer identifies connection issues (site blocked, launcher installation) for some learners and shares remediation steps and alternate cloud links. The current state: most learners connect successfully, several require additional LOD/cloud training keys that the trainer will arrange before the next session.

- QA Trainer shares individual lab links via direct message and instructs learners to copy username, password, and access code exactly to avoid lockouts.
- QA Trainer identifies two common connection blockers: the lab site being blocked by a network and the launcher installation being blocked, and outlines workarounds.
- QA Trainer asks learners who can connect to create a shortcut to the remote desktop to simplify reconnection tomorrow.
- QA Trainer commits to arranging training keys and alternate cloud desktops for six learners who cannot access the primary 'go to my PC' service.
- QA Trainer instructs the six named learners (Nicola, Lindsay, Ian, Rebecca, Nobin and one other) to redeem training keys in the student dashboard to access lab desktops.

Movie Database Modeling Details

Trainer walks the class through deriving entities and bridges from the movie dataset and demonstrates how to assign surrogate keys, choose bridge tables for many-to-many relationships, and decide which attributes belong to which entity. This matters because correct entity attribution and data types form the data dictionary and ERD that the groups must deliver; the current state is that teams understand expected entities (movie, actor, director) and two bridge tables (actor-movie-character and director-movie) and are ready to map attributes and types.

- QA Trainer identifies movie, director, and actor as the three primary entities for the exercise and instructs teams to propose two bridging tables.
- QA Trainer explains that one bridge should map actor to movie with attributes like character name and cast order so many-to-many actor-character-movie relationships are captured.
- QA Trainer advises creating a director_movie bridge where required because a director can direct many movies and some movies can have multiple directors.
- QA Trainer instructs teams to create surrogate primary keys where no key exists and to retain existing column titles for attributes coming from the collected data.
- QA Trainer specifies that bridge tables should contain only foreign keys from the related entities plus any attributes specific to the relationship, and that bridges should not introduce primary-key business attributes.

Choosing Data Types and SQL Guidance

Trainer introduces SQL data types and points learners to Microsoft T-SQL documentation, explaining why choosing correct types (numeric, date, character, unicode, binary) matters for validation and storage. The current state: learners are expected to map each attribute to an appropriate SQL type in their data dictionary using the referenced documentation.

- QA Trainer asks what data should go into surrogate keys and recommends numeric types for IDs.
- QA Trainer shares Microsoft Learn T-SQL data type references and suggests learners consult them when assigning data types.
- QA Trainer emphasizes that teams should not invent new attributes beyond the collected data except for surrogate keys and should match column titles to existing data.

SQL Data Types Deep Dive

Trainer delivers a detailed walkthrough of SQL Server data types—integers (tiny, small, standard, big), character types (varchar, nvarchar, max), decimals/numeric with precision and scale, float/real approximations, money/smallmoney, bit for boolean flags, and date/time variants—linking each type to example columns in the movie dataset. The current state: learners understand which data types suit names, budgets, popularity scores, runtimes, and dates, and that constraints (NOT NULL) and appropriate lengths should be planned in the data dictionary.

- QA Trainer recommends integer types for plain numeric IDs and explains there are sized variants (tinyint, smallint, int, bigint) based on value range and storage cost.

- QA Trainer advises using varchar or nvarchar for names and suggests picking a sensible maximum length (e.g., 50) or nvarchar when Unicode is needed.
- QA Trainer explains decimal/numeric require precision (P) and scale (S) and gives examples for vote averages where specific P and S values prevent data corruption.
- QA Trainer describes float (and real) as approximate numeric types with P values (e.g., float(53)) and notes float should be used when decimal precision varies widely, like popularity scores.
- QA Trainer states money and smallmoney exist for currency and carry storage and range considerations, and warns smallmoney can truncate values if limits are exceeded.
- QA Trainer highlights using bit for true/false logical fields and datetime/date/datetimeoffset types for temporal data, selecting variants when time or timezone precision is necessary.
- QA Trainer connects data type choice to storage implications and says technical teams handle capacity planning, but modelers must propose sensible types to avoid corruption on load.

Practical Data Type Guidance

Trainer demonstrates mapping dataset attributes to types and constraints: director/actor names as varchar/nvarchar with length and NOT NULL where required, budget/revenue as money/decimal, release dates as date/datetime, and runtime as integer. The current state: teams should apply these mappings in their data dictionaries and finalize choices in the next session when they create ERDs and complete modeling tasks.

- QA Trainer declares primary keys must be NOT NULL and recommends making name fields NOT NULL where the data is required.
- QA Trainer suggests using varchar(max) for very long text like taglines but recommends considering practical limits and using nvarchar where Unicode is needed.
- QA Trainer instructs teams to inspect Excel data for existing formats and choose types that match to avoid data corruption on import.
- QA Trainer advises teams to plan decimal precision/scale for numeric fields like vote_average and to use float when precision varies unpredictably.